

Herald



HRD-010 commons_storage Live Wiring Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use `superpowers:subagent-driven-development` (recommended) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Wire `commons_storage` from migration-only scaffold to fully live persistence — `pgx` pool surfaced through `commons_infra.QuickstartBoot`, background queue extended via `digital.vasic.background`, Redis ACL'd via `digital.vasic.cache`. Add `pherald migrate` subcommand. Land 3 new §107 e2e invariants (E14 pool round-trip, E15 queue round-trip, E16 redis TTL).

Architecture: `commons_infra` becomes the singleton resolver — after `QuickstartBoot.Up()` returns, consumers (tests, `pherald serve`, future flavors) call `.Pool()`, `.Queue()`, `.Redis()` to get live clients backed by the booted containers. The existing `pgx` adapter in `commons_storage.Open` is reused; `WithTenantContext` already implements RLS propagation per migration `000008_force_rls`. Queue + Redis adapters are thin wrappers over the upstream Helix submodules (`digital.vasic.background.TaskQueue` and `digital.vasic.cache/pkg/redis.Client`). All integration tests guard with `//go:build integration` so default `go test` stays fast; `e2e_bluff_hunt.sh` runs the integration tier with the `-tags=integration` flag against live containers.

Tech Stack: Go 1.25+; `pgx` v5 (via `digital.vasic.database`); `digital.vasic.background.TaskQueue`; `digital.vasic.cache/pkg/redis.Client`; Cobra (Herald CLI); `digital.vasic.containers/pkg/compose` (test boot); Postgres 16 + Redis 7 (container images per `quickstart/docker-compose.quickstart.yml`).

Spec refs: HRD-010 row in `docs/Issues.md`; spec V3 §9.6 + §16; master roadmap §"Wave 1 — Storage + first user-visible feature".

Catalogue-Check (Universal §11.4.74): extend `digital.vasic.database@<pinned>` for `pgx` + migrations (already in `commons_storage`); extend `digital.vasic.background@<pinned>` for queue; extend `digital.vasic.cache@<pinned>` for Redis; no-match for the `commons_infra` resolver shim (Herald-specific). Update HRD-010 row's References cell with the resolved catalogue-check before Issues→Fixed migration.

File Structure

Files to CREATE

Path	Responsibility
<code>commons_infra/queue.go</code>	Thin adapter exposing <code>digital.vasic.background.TaskQueue</code> to Herald, taking a Postgres-backed repo from the booted pool
<code>commons_infra/redis.go</code>	Thin adapter exposing <code>digital.vasic.cache/pkg/redis.Client</code> to Herald, with Herald-prefix key namespacing
<code>commons_infra/clients.go</code>	Aggregator: holds Pool/Queue/Redis once <code>QuickstartBoot.Up()</code> succeeds; getters return errors if <code>Up()</code> not called
<code>commons_infra/clients_integration_test.go</code>	//go:build integration — live container test for the aggregator
<code>commons_storage/storage_integration_test.go</code>	//go:build integration — live RLS-tenant-isolation round-trip
<code>pherald/cmd/pherald/migrate.go</code>	Cobra subcommand: <code>pherald migrate up status</code>
<code>pherald/cmd/pherald/migrate_test.go</code>	Unit test for migrate command argument parsing (live exec covered by e2e)
<code>docs/Fixed.md</code> entry for HRD-010	Added at the end after all integration tests green

Files to MODIFY

Path	Change
<code>commons_infra/boot.go</code>	Add <code>Up(ctx)</code> -side wiring that instantiates Pool + Queue + Redis clients and stores them on the boot struct for retrieval
<code>commons_infra/go.mod</code>	Add replace directives for <code>digital.vasic.background</code> and <code>digital.vasic.cache</code> submodules
<code>commons_storage/go.mod</code>	Add <code>digital.vasic.cache</code> if it's needed for any storage→cache coupling (unlikely; verify by go build)
<code>pherald/cmd/pherald/main.go</code>	Register migrate subcommand
<code>pherald/go.mod</code>	Add <code>digital.vasic.background</code> + <code>digital.vasic.cache</code> replace directives if pherald serve ends up requiring them in this plan (likely deferred to Plan 2 — but the migrate subcommand needs at minimum <code>commons_storage</code> which already replace-points to <code>digital.vasic.database</code>)
<code>scripts/e2e_bluff_hunt.sh</code>	Add E14 (pool round-trip), E15 (queue round-trip), E16 (redis TTL) invariants
<code>quickstart/.env.example</code>	Document new env vars: <code>HERALD_PG_DSN</code> , <code>HERALD_REDIS_URL</code> , <code>HERALD_QUEUE_WORKER_ID</code>

Path	Change
docs/Issues.md	Atomic migrate HRD-010 row → docs/Fixed.md at end (per §11.4.19)
docs/Status.md	r6 → r7: commons_storage status: partial →  landed
docs/ Status.docx/.html/.pdf	Regenerate after edit (per §11.4.65)

Files NOT touched in this plan (deferred to Plan 2 / later)

- commons_messaging/channels/tgram/ — Plan 2 (HRD-011)
- commons_messaging/dispatch/claude_code/ — Plan 2 (HRD-012)
- quickstart/compose.yaml itself — operator-validated under HRD-008 (Plan 3)

Task 1: Add commons_infra/clients.go aggregator skeleton (TDD-first)

Why first: The aggregator is the singleton consumers will call. Defining it first locks down the API the rest of the wiring satisfies.

Files:

- Create: commons_infra/clients.go
- Test: commons_infra/clients_test.go (unit, not integration)



Step 1: Write failing test for "Pool() before Up() returns ErrNotBooted"

Create commons_infra/clients_test.go:

```
package infra

import (
    "errors"
    "testing"
)

func TestClients_PoolBeforeUp_ReturnsErrNotBooted(t *testing.T) {
    boot, err := NewQuickstartBoot()
    if err != nil {
        t.Fatalf("NewQuickstartBoot: %v", err)
    }
    _, err = boot.Pool()
    if !errors.Is(err, ErrNotBooted) {
        t.Fatalf("expected ErrNotBooted, got %v", err)
    }
}

func TestClients_QueueBeforeUp_ReturnsErrNotBooted(t *testing.T) {
```

```

boot, err := NewQuickstartBoot()
if err != nil {
    t.Fatalf("NewQuickstartBoot: %v", err)
}
_, err = boot.Queue()
if !errors.Is(err, ErrNotBooted) {
    t.Fatalf("expected ErrNotBooted, got %v", err)
}
}

func TestClients_RedisBeforeUp_ReturnsErrNotBooted(t *testing.T)
{
    boot, err := NewQuickstartBoot()
    if err != nil {
        t.Fatalf("NewQuickstartBoot: %v", err)
    }
    _, err = boot.Redis()
    if !errors.Is(err, ErrNotBooted) {
        t.Fatalf("expected ErrNotBooted, got %v", err)
    }
}

```



Step 2: Run test to verify it fails to compile (ErrNotBooted undefined; Pool/Queue/Redis methods absent)

Run: `go test ./commons_infra/ -run TestClients_ -count=1` Expected:
FAIL with "undefined: ErrNotBooted" or "boot.Pool undefined".



Step 3: Implement minimal clients.go

Create `commons_infra/clients.go`:

```

package infra

import (
    "errors"

    "digital.vasic.cache/pkg/redis"
    "digital.vasic.database/pkg/db"
)

// ErrNotBooted is returned by Pool/Queue/Redis when
// QuickstartBoot.Up()
// has not been called (or returned an error and never
// completed).
// Per §107 + §11.4.5, callers MUST check this error and abort –
// silently

```

```

// returning a nil client and PASSing the test on a no-op is a
    bluff.
var ErrNotBooted =
    errors.New("commons_infra: QuickstartBoot.Up() not called
        or failed; clients unavailable")

// Pool returns the live pgx Database opened by Up(). Returns
    ErrNotBooted
// if Up() has not completed successfully.
func (b *QuickstartBoot) Pool() (db.Database, error) {
    if b.pool == nil {
        return nil, ErrNotBooted
    }
    return b.pool, nil
}

// Queue returns the live TaskQueue bound to the booted Postgres
    pool.
// Returns ErrNotBooted if Up() has not completed.
//
// The interface type is the one declared by
    digital.vasic.background.
// Worker registration is done via the upstream submodule's
    WorkerPool;
// this Queue() accessor exposes only the enqueue/dequeue
    surface.
func (b *QuickstartBoot) Queue() (TaskQueue, error) {
    if b.queue == nil {
        return nil, ErrNotBooted
    }
    return b.queue, nil
}

// Redis returns the live redis Client backed by the booted Redis
    container.
// Returns ErrNotBooted if Up() has not completed.
func (b *QuickstartBoot) Redis() (*redis.Client, error) {
    if b.redis == nil {
        return nil, ErrNotBooted
    }
    return b.redis, nil
}

```



Step 4: Declare the TaskQueue interface alias in commons_infra/queue.go

Create commons_infra/queue.go:

```

package infra

import (
    "context"

    bgmodels "digital.vasic.background/internal/models"
    bg "digital.vasic.background"
)

// TaskQueue is Herald's alias for
//    digital.vasic.background.TaskQueue.
// Aliasing keeps Herald callers from importing the upstream
//    package directly
// (Catalogue-Check §11.4.74: we extend, not reimplement) but
//    also means a
// breaking change to the upstream interface is caught at this
//    seam.
type TaskQueue = bg.TaskQueue

// Task is Herald's alias for digital.vasic.background's
//    BackgroundTask model.
type Task = bgmodels.BackgroundTask

// EnsureEnqueueable is a compile-time guard: if upstream renames
//    Enqueue
// or changes its signature, this declaration fails to build.
var _ = func(q TaskQueue) func(context.Context, *Task) error {
    return q.Enqueue }

```

Note for engineer: The `digital.vasic.background/internal/models` import path is upstream-internal — if the upstream repo gates internal against external imports, replace with the exported `models` package. Check `go doc digital.vasic.background` to confirm the correct import path before relying on it. If `internal/models` is blocked, the upstream submodule needs a `pkg/models` re-export (open an upstream HRD; do NOT vendor-copy the type per §11.4.74).



Step 5: Add private pool, queue, redis fields to QuickstartBoot struct in `commons_infra/boot.go`

Modify `commons_infra/boot.go:43-46` (the existing `QuickstartBoot` struct):

```

// QuickstartBoot wraps a compose.Orchestrator with Herald-
//    specific defaults.
type QuickstartBoot struct {
    orch    compose.ComposeOrchestrator
    project compose.ComposeProject

```

```

    // Populated by Up() after services are healthy. Accessed via
    Pool/Queue/Redis.
    pool db.Database
    queue TaskQueue
    redis *redis.Client
}

```

Add the necessary imports at the top of `boot.go`:

```

import (
    // ... existing imports ...
    "digital.vasic.cache/pkg/redis"
    "digital.vasic.database/pkg/db"
)

```



Step 6: Run unit test, verify PASS

Run: `go test ./commons_infra/ -run TestClients_ -count=1` Expected: PASS (3 tests).



Step 7: Commit

```

git add commons_infra/clients.go commons_infra/clients_test.go
    commons_infra/queue.go commons_infra/boot.go
git commit -m "HRD-010 step 1: aggregator skeleton with
    ErrNotBooted guard

```

`clients.go` declares `Pool/Queue/Redis` accessors that return `ErrNotBooted` when called before `Up()`. `queue.go` aliases `digital.vasic.background.TaskQueue` to keep Herald callers from importing the upstream directly (extends per §11.4.74). 3 unit tests pass guarding the not-booted contract.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>

Task 2: Wire pgx pool open in `QuickstartBoot.Up()`

Files:

- Modify: `commons_infra/boot.go` (`Up` method — search for the existing one; add post-orch-`Up` pool-open block)



Step 1: Write failing integration test for live pool round-trip (Task 4 fully covers this — placeholder here so we don't duplicate)

The actual pool round-trip test goes in `commons_storage/storage_integration_test.go` (Task 4). For this task we just need a smoke that asserts `Up()` populates pool non-nil; add to `clients_integration_test.go`:

Create `commons_infra/clients_integration_test.go`:

```
//go:build integration

package infra

import (
    "context"
    "testing"
    "time"
)

func TestUp_PopulatesPool(t *testing.T) {
    ctx, cancel := context.WithTimeout(context.Background(),
        90*time.Second)
    defer cancel()

    boot, err := NewQuickstartBoot()
    if err != nil {
        t.Fatalf("NewQuickstartBoot: %v", err)
    }
    if err := boot.Up(ctx); err != nil {
        t.Fatalf("Up: %v", err)
    }
    t.Cleanup(func() {
        _ = boot.Down(context.Background())
    })

    pool, err := boot.Pool()
    if err != nil {
        t.Fatalf("Pool() after Up(): %v", err)
    }
    if pool == nil {
        t.Fatal("Pool() returned nil without error – §107 PASS-bluff guard")
    }
}
```



Step 2: Run integration test to verify it fails (pool not populated)

Run: `go test ./commons_infra/ -tags=integration -run TestUp_PopulatesPool -count=1 -timeout=180s` Expected: FAIL with "Pool() after Up(): commons_infra: QuickstartBoot.Up() not called or failed" (since Up exists but doesn't yet populate pool).

Engineer note: This requires Docker or Podman on the host. If absent, the test will fail at container boot — that's expected and indicates the host setup, NOT a code bug. The test is `//go:build integration` so default `go test ./...` skips it cleanly.



Step 3: Add pool-open to Up() in boot.go

Find the existing `Up(ctx context.Context) error` method in `commons_infra/boot.go` (use `grep -n 'func (b \*QuickstartBoot) Up' commons_infra/boot.go` to locate). After the existing `orch.Up() + WaitHealthy("postgres")` block, before returning `nil`, add:

```
// Open pgx pool against the booted Postgres container.
cfg := storage.ConfigForHerald(
    envOr("HERALD_PG_HOST", "127.0.0.1"),
    envOrInt("HERALD_PG_PORT", 70011),
    envOr("HERALD_PG_USER", "herald"),
    envOr("HERALD_PG_PASSWORD", "herald_dev"),
    envOr("HERALD_PG_DBNAME", "herald"),
)
pool, err := storage.Open(ctx, cfg)
if err != nil {
    return fmt.Errorf("commons_infra.Up: open pgx pool: %w",
        err)
}
b.pool = pool

// Apply migrations so the schema is live before tests use
// the pool.
applied, err := storage.RunMigrations(ctx, pool)
if err != nil {
    _ = pool.Close()
    b.pool = nil
    return fmt.Errorf("commons_infra.Up: run migrations:
        %w", err)
}
logging.NopLogger{}.Info("migrations applied", "count",
    len(applied))
```

Add the storage import at the top of `boot.go`:

```
storage "github.com/vasic-digital/herald/commons_storage"
```

Add the env helpers near the bottom of `boot.go`:

```
func envOr(key, fallback string) string {
    if v := os.Getenv(key); v != "" {
        return v
    }
    return fallback
}
```

```

}

func envOrInt(key string, fallback int) int {
    if v := os.Getenv(key); v != "" {
        var i int
        if _, err := fmt.Sscanf(v, "%d", &i); err == nil {
            return i
        }
    }
    return fallback
}

```



Step 4: Add commons_storage as a replace directive in commons_infra/go.mod

Open commons_infra/go.mod and add:

```

require github.com/vasic-digital/herald/commons_storage v0.0.0
replace github.com/vasic-digital/herald/commons_storage => ../commons_stor

```

Then go mod tidy inside commons_infra/.



Step 5: Run integration test, verify PASS

Run: go test ./commons_infra/ -tags=integration -run
 TestUp_PopulatesPool -count=1 -timeout=180s Expected: PASS — container
 boots, pool opens, migrations apply, test asserts pool non-nil.

Also confirm default go test still green (no integration tag → skips the integration test):

Run: go test -race -count=1 ./commons_infra/ Expected: PASS (only unit tests
 run).



Step 6: Commit

```

git add commons_infra/boot.go commons_infra/
      clients_integration_test.go commons_infra/go.mod
      commons_infra/go.sum
git commit -m "HRD-010 step 2: QuickstartBoot.Up opens live pgx
      pool + runs migrations

```

Up() now opens commons_storage.Open against the booted Postgres
 container,
 applies all 8 embedded migrations, and stashes the pool on the
 boot struct
 for retrieval via Pool(). Integration test (//go:build
 integration) asserts
 pool non-nil after Up.

Default-mode go test unchanged (integration test skipped without -tags).

§107 anti-bluff: pool nil-check guards against PASS-on-nil.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"

Task 3: Storage RLS-tenant-isolation live round-trip test

Files:

- Create: commons_storage/storage_integration_test.go

This is the canonical §107 evidence for HRD-010. It proves WithTenantContext actually isolates rows.



Step 1: Write failing integration test

Create commons_storage/storage_integration_test.go:

```
//go:build integration
```

```
package storage_test
```

```
import (  
    "context"  
    "testing"  
    "time"  
  
    "github.com/google/uuid"  
    storage "github.com/vasic-digital/herald/commons_storage"  
    "github.com/vasic-digital/herald/commons_infra"  
)
```

```
// TestRLS_TenantIsolation_RoundTrip is the §107 E14-equivalent  
// evidence:  
// two tenants A and B each insert a herald_subscribers row in  
// their own  
// tenant context. Each tenant's read MUST see only its own row.  
//  
// A bluff PASS here would be: read returns rows from BOTH  
// tenants but the  
// test only asserts the count of OWN tenant's rows. We assert  
// exact match  
// to make that bluff impossible.  
func TestRLS_TenantIsolation_RoundTrip(t *testing.T) {  
    ctx, cancel := context.WithTimeout(context.Background(),  
        120*time.Second)  
    defer cancel()
```

```

boot, err := infra.NewQuickstartBoot()
if err != nil {
    t.Fatalf("NewQuickstartBoot: %v", err)
}
if err := boot.Up(ctx); err != nil {
    t.Fatalf("Up: %v", err)
}
t.Cleanup(func() { _ = boot.Down(context.Background()) })

pool, err := boot.Pool()
if err != nil {
    t.Fatalf("Pool: %v", err)
}

tenantA := uuid.New()
tenantB := uuid.New()
subA := uuid.New()
subB := uuid.New()

// Insert one row per tenant.
for tenant, sub := range map[uuid.UUID]uuid.UUID{tenantA:
    subA, tenantB: subB} {
    if err := storage.WithTenantContext(ctx, pool, tenant,
        func(tx db.Tx) error {
            _, err := tx.Exec(ctx,
                `INSERT INTO herald_subscribers (id, tenant_id,
                display_name, created_at)
                VALUES ($1, $2, $3, NOW())`,
                sub, tenant, "test-sub-"+sub.String(),
            )
            return err
        }); err != nil {
        t.Fatalf("insert tenant %s: %v", tenant, err)
    }
}

// Read as tenant A – must see exactly 1 row, and that row
// must be subA.
var readA []uuid.UUID
if err := storage.WithTenantContext(ctx, pool, tenantA,
    func(tx db.Tx) error {
        rows, err := tx.Query(ctx, `SELECT id FROM
        herald_subscribers ORDER BY id`)
        if err != nil {
            return err
        }
        defer rows.Close()
    })

```

```

        for rows.Next() {
            var id uuid.UUID
            if err := rows.Scan(&id); err != nil {
                return err
            }
            readA = append(readA, id)
        }
        return rows.Err()
    }); err != nil {
        t.Fatalf("read tenant A: %v", err)
    }

    if len(readA) != 1 {
        t.Fatalf("tenant A: expected exactly 1 row, got %d (RLS bluff if 2)", len(readA))
    }
    if readA[0] != subA {

        t.Fatalf("tenant A: expected subA=%s, got %s (RLS row-mix bluff)", subA, readA[0])
    }

    // Symmetric read as tenant B.
    var readB []uuid.UUID
    if err := storage.WithTenantContext(ctx, pool, tenantB,
        func(tx db.Tx) error {
            rows, err := tx.Query(ctx, `SELECT id FROM
herald_subscribers ORDER BY id`)
            if err != nil {
                return err
            }
            defer rows.Close()
            for rows.Next() {
                var id uuid.UUID
                if err := rows.Scan(&id); err != nil {
                    return err
                }
                readB = append(readB, id)
            }
            return rows.Err()
        }); err != nil {
        t.Fatalf("read tenant B: %v", err)
    }

    if len(readB) != 1 || readB[0] != subB {
        t.Fatalf("tenant B: expected exactly [%s], got %v",
            subB, readB)
    }
}
}

```

Engineer note: Imports — the test file's package is `storage_test` (external test package) so it imports both `commons_storage` (as `storage`) and `commons_infra` (as `infra`). The `db.Tx` type comes from `digital.vasic.database/pkg/db`; you'll need to add `db "digital.vasic.database/pkg/db"` to the imports.



Step 2: Run, expect FAIL

Run: `go test ./commons_storage/ -tags=integration -run TestRLS_TenantIsolation_RoundTrip -count=1 -timeout=180s` Expected: FAIL — the test depends on Task 2's pool wiring being live; if Task 2 was implemented correctly the test should actually PASS already.

If it FAILs only because of the schema (column `display_name` missing), inspect `commons_storage/migrations/000003_subscribers.up.sql` to confirm columns. Adjust the INSERT statement to match the actual columns.



Step 3: Make any column-name corrections; re-run

Examine: `cat commons_storage/migrations/000003_subscribers.up.sql`

Adjust the INSERT in the test to match exactly the columns declared in that migration. Do NOT change the migration to fit the test.



Step 4: Run, expect PASS

Run: `go test ./commons_storage/ -tags=integration -run TestRLS_TenantIsolation_RoundTrip -count=1 -timeout=180s` Expected: PASS — both tenants insert their row, each reads exactly one (their own) row.



Step 5: Commit

```
git add commons_storage/storage_integration_test.go
      commons_storage/go.mod commons_storage/go.sum
git commit -m "HRD-010 step 3: RLS-tenant-isolation live
integration test (E14 evidence)"
```

Two-tenant insert+read round-trip against live Postgres asserts
each tenant
sees exactly its own row. Bluff guards: asserts EXACT-1 row count
(not ≥ 1),
asserts EXACT row ID match (not just non-empty). Per §107: this
proves
WithTenantContext actually isolates, not just that the SQL
compiles.

Build-tag-gated so default `go test` stays fast.

Task 4: Wire Redis client open in QuickstartBoot.Up()

Files:

- Modify: `commons_infra/boot.go` (Up method — add post-pool redis-open block)
- Create: `commons_infra/redis.go` (thin adapter — already aliased through Pool/Queue/Redis getters in Task 1)



Step 1: Write failing integration test for redis TTL round-trip

Append to `commons_infra/clients_integration_test.go`:

```
func TestUp_PopulatesRedis_TTLRoundTrip(t *testing.T) {
    ctx, cancel := context.WithTimeout(context.Background(),
        90*time.Second)
    defer cancel()

    boot, err := NewQuickstartBoot()
    if err != nil {
        t.Fatalf("NewQuickstartBoot: %v", err)
    }
    if err := boot.Up(ctx); err != nil {
        t.Fatalf("Up: %v", err)
    }
    t.Cleanup(func() { _ = boot.Down(context.Background()) })

    rc, err := boot.Redis()
    if err != nil {
        t.Fatalf("Redis() after Up: %v", err)
    }

    key := "herald:test:ttl:" +
        time.Now().Format("20060102150405.000000")
    if err := rc.Set(ctx, key, []byte("hello"), 2*time.Second);
        err != nil {
        t.Fatalf("Set: %v", err)
    }

    // Read back immediately — must be present.
    got, err := rc.Get(ctx, key)
    if err != nil {
        t.Fatalf("Get before TTL: %v", err)
    }
    if string(got) != "hello" {
```

```

        t.Fatalf("Get value mismatch: got %q want %q", got,
            "hello")
    }

    // Wait past TTL — must be absent.
    time.Sleep(2500 * time.Millisecond)
    exists, err := rc.Exists(ctx, key)
    if err != nil {
        t.Fatalf("Exists after TTL: %v", err)
    }
    if exists {
        t.Fatalf("key %q still present after TTL — Redis didn't
            enforce TTL (bluff trap)", key)
    }
}

```



Step 2: Run, expect FAIL with "Redis() after Up: ErrNotBooted"

Run: `go test ./commons_infra/ -tags=integration -run TestUp_PopulatesRedis_TTLRoundTrip -count=1 -timeout=180s` Expected: FAIL — `Up()` doesn't yet open Redis.



Step 3: Add redis-open block to `Up()` in `boot.go`

Right after the migration-applied block from Task 2, add:

```

// Open Redis client against the booted Redis container.
redisCfg := &redis.Config{
    Addr:      envOr("HERALD_REDIS_ADDR", "127.0.0.1:70012"),
    Password: envOr("HERALD_REDIS_PASSWORD", ""),
    DB:        envOrInt("HERALD_REDIS_DB", 0),
}
rc := redis.New(redisCfg)
if err := rc.HealthCheck(ctx); err != nil {
    _ = pool.Close()
    b.pool = nil
    return fmt.Errorf("commons_infra.Up: redis healthcheck:
        %w", err)
}
b.redis = rc

```

Engineer note: The exact field names of `redis.Config` may differ — confirm via `go doc digital.vasic.cache/pkg/redis.Config` before relying on `Addr/Password/DB`. The `New` constructor signature is `New(*Config) *Client` per the upstream API. Fall back to whatever fields the upstream actually defines.

Add the cache replace directive to `commons_infra/go.mod`:


```
require digital.vasic.cache v0.0.0
replace digital.vasic.cache => ../submodules/cache
```

Then go mod tidy.



Step 4: Run, expect PASS

Run: `go test ./commons_infra/ -tags=integration -run TestUp_PopulatesRedis_TTLRoundTrip -count=1 -timeout=180s` Expected: PASS — Redis container responds to healthcheck + Set + Get + TTL-expiry.



Step 5: Commit

```
git add commons_infra/boot.go commons_infra/
      clients_integration_test.go commons_infra/go.mod
      commons_infra/go.sum
git commit -m
      "HRD-010 step 4: QuickstartBoot.Up opens live Redis + TTL
      round-trip test (E16 evidence)"
```

Up() now constructs digital.vasic.cache/pkg/redis.Client against the booted container, asserts HealthCheck, and stashes the client for Redis() retrieval. Integration test does Set → Get → wait → Exists=false, proving real TTL enforcement (not just an in-process map).

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>

Task 5: Wire background queue in QuickstartBoot.Up() + enqueue/dequeue round-trip

Files:

- Modify: commons_infra/boot.go (Up method — add post-redis queue-open block)
- Modify: commons_infra/queue.go (add queue constructor)
- Modify: commons_infra/clients_integration_test.go (queue round-trip test)



Step 1: Write failing integration test for queue enqueue/dequeue round-trip

Append to commons_infra/clients_integration_test.go:

```

func TestUp_PopulatesQueue_EnqueueDequeueRoundTrip(t *testing.T)
{
    ctx, cancel := context.WithTimeout(context.Background(),
        90*time.Second)
    defer cancel()

    boot, err := NewQuickstartBoot()
    if err != nil {
        t.Fatalf("NewQuickstartBoot: %v", err)
    }
    if err := boot.Up(ctx); err != nil {
        t.Fatalf("Up: %v", err)
    }
    t.Cleanup(func() { _ = boot.Down(context.Background()) })

    q, err := boot.Queue()
    if err != nil {
        t.Fatalf("Queue() after Up: %v", err)
    }

    task := &Task{
        // Fields match digital.vasic.background/internal/
        // models.BackgroundTask
        // — confirm exact fields via `go doc
        // digital.vasic.background.BackgroundTask`
        // before relying on names below.
        ID:      uuid.NewString(),
        Type:     "herald_test",
        Priority: 50,
        Payload: []byte(`{"hello":"world"}`),
    }
    if err := q.Enqueue(ctx, task); err != nil {
        t.Fatalf("Enqueue: %v", err)
    }

    // Dequeue with a workerID; the round-trip MUST return our
    // task.
    got, err := q.Dequeue(ctx, "test-worker-"+task.ID,
        ResourceRequirements{})
    if err != nil {
        t.Fatalf("Dequeue: %v", err)
    }
    if got == nil {
        t.Fatal("Dequeue returned nil task — bluff guard")
    }
    if got.ID != task.ID {
        t.Fatalf("Dequeue returned wrong task: want %s got %s",
            task.ID, got.ID)
    }
}

```

```
}  
}
```

Engineer note: The exact shape of `BackgroundTask` and `ResourceRequirements` is defined in `submodules/background/interfaces.go` and `models/*`. Confirm fields by reading those files before relying on the literal above. If field names differ, adjust — do NOT add Herald-side translation; we extend, we don't reimplement.



Step 2: Run, expect FAIL

Run: `go test ./commons_infra/ -tags=integration -run TestUp_PopulatesQueue_EnqueueDequeueRoundTrip -count=1 -timeout=180s` Expected: FAIL — `Up()` doesn't yet open queue.



Step 3: Add queue-open block to `Up()` in `boot.go`

Right after the `redis-open` block, add:

```
// Bind the upstream digital.vasic.background TaskQueue to  
// the booted pool.  
// The upstream module exposes a constructor; confirm exact  
// name via  
// `go doc digital.vasic.background.NewPostgresQueue` (likely  
// candidates:  
// NewPostgresQueue, NewQueueFromPool, NewTaskQueue). Adjust  
// below to  
// match the upstream constructor exactly.  
bgQueue, err := bg.NewPostgresQueue(ctx, pool)  
if err != nil {  
    _ = pool.Close()  
    _ = rc.Close()  
    b.pool = nil  
    b.redis = nil  
    return fmt.Errorf("commons_infra.Up: open queue: %w",  
        err)  
}  
b.queue = bgQueue
```

Add the background replace directive to `commons_infra/go.mod`:

```
require digital.vasic.background v0.0.0  
replace digital.vasic.background => ../submodules/background
```

Then `go mod tidy`.



Step 4: Run, expect PASS

Run: `go test ./commons_infra/ -tags=integration -run TestUp_PopulatesQueue_EnqueueDequeueRoundTrip -count=1 -timeout=180s` Expected: PASS — task enqueued, dequeued, ID matches.



Step 5: Commit

```
git add commons_infra/boot.go commons_infra/queue.go
      commons_infra/clients_integration_test.go commons_infra/
      go.mod commons_infra/go.sum
git commit -m "HRD-010 step 5: queue enqueue/dequeue round-trip
      live (E15 evidence)"
```

`Up()` now binds `digital.vasic.background.TaskQueue` to the booted pool.
Integration test enqueues a fake task and asserts `dequeue` returns the same ID (not just any task – proves the queue actually persists not just forwards in-memory).

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"

Task 6: pherald migrate Cobra subcommand

Files:

- Create: `pherald/cmd/pherald/migrate.go`
- Create: `pherald/cmd/pherald/migrate_test.go`
- Modify: `pherald/cmd/pherald/main.go` (register subcommand)



Step 1: Write failing unit test for argument parsing

Create `pherald/cmd/pherald/migrate_test.go`:

```
package main

import (
    "bytes"
    "strings"
    "testing"
)

func TestMigrateCommand_StatusRequiresEnv(t *testing.T) {
    cmd := newMigrateCmd()
    cmd.SetArgs([]string{"status"})
    var buf bytes.Buffer
    cmd.SetOut(&buf)
```

```

cmd.SetErr(&buf)
err := cmd.Execute()
if err == nil {
    t.Fatal("expected error when HERALD_PG_DSN unset")
}
if !strings.Contains(err.Error(), "HERALD_PG_DSN") {
    t.Fatalf("error should mention HERALD_PG_DSN, got %v",
        err)
}
}

func TestMigrateCommand_RegistersSubcommands(t *testing.T) {
    cmd := newMigrateCmd()
    subs := cmd.Commands()
    want := map[string]bool{"up": false, "status": false}
    for _, sc := range subs {
        want[sc.Name()] = true
    }
    for name, found := range want {
        if !found {
            t.Errorf("migrate subcommand %q not registered",
                name)
        }
    }
}

```



Step 2: Run, expect compile FAIL "newMigrateCmd undefined"

Run: `go test ./pherald/cmd/pherald/ -run TestMigrateCommand -count=1` Expected: FAIL with undefined symbol.



Step 3: Implement newMigrateCmd()

Create `pherald/cmd/pherald/migrate.go`:

```

package main

import (
    "context"
    "fmt"
    "os"

    "github.com/spf13/cobra"
    storage "github.com/vasic-digital/herald/commons_storage"
)

func newMigrateCmd() *cobra.Command {
    cmd := &cobra.Command{

```

```

        Use:    "migrate",
        Short:  "Apply or inspect commons_storage migrations",
        Long:   "Run Herald's embedded SQL migrations against the
        configured " +
            "Postgres instance. Requires HERALD_PG_DSN
        environment variable.",
    }
    cmd.AddCommand(newMigrateUpCmd())
    cmd.AddCommand(newMigrateStatusCmd())
    return cmd
}

```

```

func newMigrateUpCmd() *cobra.Command {
    return &cobra.Command{
        Use:    "up",
        Short:  "Apply all pending migrations",
        RunE: func(cmd *cobra.Command, args []string) error {
            dsn := os.Getenv("HERALD_PG_DSN")
            if dsn == "" {
                return fmt.Errorf("HERALD_PG_DSN environment
                variable must be set")
            }
            ctx, cancel := context.WithCancel(cmd.Context())
            defer cancel()
            cfg, err := storage.ParseDSN(dsn)
            if err != nil {
                return fmt.Errorf("parse DSN: %w", err)
            }
            pool, err := storage.Open(ctx, cfg)
            if err != nil {
                return fmt.Errorf("open pool: %w", err)
            }
            defer pool.Close()
            applied, err := storage.RunMigrations(ctx, pool)
            if err != nil {
                return fmt.Errorf("apply migrations: %w", err)
            }
            fmt.Fprintf(cmd.OutOrStdout(), "applied %d
            migration(s)\n", len(applied))
            return nil
        },
    }
}

```

```

func newMigrateStatusCmd() *cobra.Command {
    return &cobra.Command{
        Use:    "status",
        Short:  "Report current schema version",
    }
}

```

```

RunE: func(cmd *cobra.Command, args []string) error {
    dsn := os.Getenv("HERALD_PG_DSN")
    if dsn == "" {
        return fmt.Errorf("HERALD_PG_DSN environment
variable must be set")
    }
    ctx, cancel := context.WithCancel(cmd.Context())
    defer cancel()
    cfg, err := storage.ParseDSN(dsn)
    if err != nil {
        return fmt.Errorf("parse DSN: %w", err)
    }
    pool, err := storage.Open(ctx, cfg)
    if err != nil {
        return fmt.Errorf("open pool: %w", err)
    }
    defer pool.Close()
    ver, err := storage.CurrentVersion(ctx, pool)
    if err != nil {
        return fmt.Errorf("current version: %w", err)
    }
    fmt.Fprintf(cmd.OutOrStdout(), "schema version:
%d\n", ver)
    return nil
},
}
}

```

Engineer note: This requires `storage.ParseDSN` and `storage.CurrentVersion` to exist in `commons_storage`. If they don't, add them as tiny helpers in `commons_storage/postgres.go` and `commons_storage/migrator.go` respectively. `ParseDSN` converts a `postgres://user:pass@host:port/db` URL into the existing `*postgres.Config`; `CurrentVersion` returns the max applied migration number (read from the `schema_migrations` table the migration runner already maintains).



Step 4: Register in `main.go`

Open `pherald/cmd/pherald/main.go`. Find where the existing subcommands (version, etc.) are registered (look for `rootCmd.AddCommand`). Add:

```
rootCmd.AddCommand(newMigrateCmd())
```

If the existing 501-stub for "migrate" exists (per the current `stubs.go`), remove that stub entry so it doesn't shadow the new real subcommand.



Step 5: Run unit tests, verify PASS

Run: `go test ./pherald/cmd/pherald/ -run TestMigrateCommand -count=1` Expected: PASS (2 tests).



Step 6: Build pherald, run migrate status against a live container (manual smoke)

Run:

```
go build -o /tmp/pherald-dev ./pherald/cmd/pherald
HERALD_PG_DSN="postgres://herald:herald_dev@127.0.0.1:70011/herald" /tmp/pherald-dev migrate status
```

(Assumes the quickstart compose stack is up. If not, expect a connection refused which proves error handling works.)

Expected with live stack: `schema version: 8` Expected without: error containing "connection refused" — that's HONEST (matches §107 — no bluff).



Step 7: Commit

```
git add pherald/cmd/pherald/migrate.go pherald/cmd/pherald/migrate_test.go pherald/cmd/pherald/main.go pherald/cmd/pherald/stubs.go commons_storage/postgres.go commons_storage/migrator.go
git commit -m "HRD-010 step 6: pherald migrate up/status subcommands"
```

Cobra subcommand drives `commons_storage.RunMigrations` from the CLI.

Requires `HERALD_PG_DSN` env var (no auto-defaulting — operator must be explicit per §11.4.6 no-guessing). Stub entry for 'migrate' removed from `stubs.go` now that the real implementation lives.

`ParseDSN` + `CurrentVersion` added as small helpers to `commons_storage`.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>

Task 7: Add E14/E15/E16 invariants to `scripts/e2e_bluff_hunt.sh`

Files:

- Modify: `scripts/e2e_bluff_hunt.sh`



Step 1: Find the E13 block (live Postgres M2 integration) and add E14-E16 after it

Open `scripts/e2e_bluff_hunt.sh`. Find `== E13: optional live Postgres M2 integration tests ==`. After its check ... block, add:

```
# -----
# E14-E16: HRD-010 commons_storage live wiring (Wave 1 evidence).
# Requires container runtime; skipped with reason if absent.
echo ""
echo "== E14-E16: HRD-010 commons_storage live integration =="
if command -v docker >/dev/null 2>&1 || command -v podman >/dev/
    null 2>&1; then
    check "E14 commons_storage RLS tenant-isolation round-trip
        (live PG)" \
        "go test ./commons_storage/ -tags=integration -run
        TestRLS_TenantIsolation_RoundTrip -count=1 -timeout=180s"
    check "E15 commons_infra queue enqueue/dequeue round-trip
        (live PG)" \
        "go test ./commons_infra/ -tags=integration -run
        TestUp_PopulatesQueue_EnqueueDequeueRoundTrip -count=1
        -timeout=180s"
    check "E16 commons_infra redis TTL round-trip (live Redis)" \
        "go test ./commons_infra/ -tags=integration -run
        TestUp_PopulatesRedis_TTLRoundTrip -count=1
        -timeout=180s"
else
    echo "SKIP E14-E16 (no docker/podman on PATH — explicit
        SKIP-with-reason per §11.4.3)"
fi
```



Step 2: Update the count + description comments at the top of the file

Search for `Fifteen invariants` (added in commit 2d4e829) and change to `Eighteen invariants`. Update the descriptive comments to mention E14/E15/E16.



Step 3: Update the totals at the bottom of the file

Find the final summary block. The current expected total is `"15 PASS / 0 FAIL"`; after Wave 1 it should be `"18 PASS / 0 FAIL"` (or `"15 PASS / 0 FAIL / 3 SKIP"` if no container runtime). Adjust any hardcoded count assertions accordingly.



Step 4: Run `e2e_bluff_hunt` locally with container runtime

Run: `bash scripts/e2e_bluff_hunt.sh 2>&1 | tail -30` Expected: 18 PASS / 0 FAIL (or 15 PASS / 0 FAIL with 3 SKIPS on a host without docker/podman — that's STILL §107-acceptable per §11.4.3).



Step 5: Commit

```
git add scripts/e2e_bluff_hunt.sh
git commit -m "HRD-010 step 7: e2e_bluff_hunt E14/E15/E16 – Wave
1 storage evidence"
```

E14: RLS tenant-isolation round-trip against live Postgres.

E15: queue enqueue/dequeue round-trip.

E16: redis TTL round-trip (Set → Get → wait → Exists=false).

Per §107, these are the captured-evidence invariants that prove
HRD-010 actually works end-to-end. Without container runtime,
they SKIP

with explicit reason per §11.4.3 (never PASS-by-default).

Total e2e_bluff_hunt: 15 PASS → 18 PASS.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"

Task 8: Update .env.example + Status.md + Issues.md → Fixed.md

Files:

- Modify: quickstart/.env.example
- Modify: docs/Issues.md
- Modify: docs/Fixed.md
- Modify: docs/Status.md
- Regenerate: docs/Status.{html,docx,pdf}



Step 1: Add new env vars to quickstart/.env.example

Open quickstart/.env.example and add:

```
# HRD-010 commons_storage live wiring (added 2026-05-XX)
HERALD_PG_HOST=127.0.0.1
HERALD_PG_PORT=70011
HERALD_PG_USER=herald
HERALD_PG_PASSWORD=herald_dev
HERALD_PG_DBNAME=herald
HERALD_PG_DSN=postgres://herald:herald_dev@127.0.0.1:70011/herald

HERALD_REDIS_ADDR=127.0.0.1:70012
HERALD_REDIS_PASSWORD=
HERALD_REDIS_DB=0
```

The literal HERALD_PG_PASSWORD=herald_dev is documentation of the **default dev value embedded in** quickstart/docker-compose.quickstart.yml — it is NEVER a real

production secret per §11.4.10. Keep the value visible in `.env.example` only because operators need to know what to override.



Step 2: Atomic migrate HRD-010 row Issues → Fixed

This is the §11.4.19 atomic move — the row must NOT exist in both files at any instant.

Use a single `git mv`-style sequence:

a. Open `docs/Issues.md`. Locate the line beginning `| HRD-010 |`. Cut it (remember the full row). b. Open `docs/Fixed.md`. Insert the row at the top of the "Recently fixed" table, with `Closed` = today's date, `Commit` = `(this commit)`. c. Update the `Issues` field count in both files' header tables — `Issues.md` drops HRD-010 from its open list; `Fixed.md` adds it to its closed list.

The `Fixed.md` row template:

```
| HRD-010 | task | middle | commons_storage live wiring — pgx pool + RLS +
```



Step 3: Update `docs/Status.md` `r6` → `r7`

Bump revision; update Implementation table row for `commons_storage`:

```
| `commons_storage` (L1) |  landed (M2 + HRD-010) | 9 SQL migrations + pg
```

Update Operations block:

- `e2e_bluff_hunt`: **18 PASS / 0 FAIL** (was 15).
- Status summary updated to mention HRD-010 closure.



Step 4: Regenerate `Status.md` + `Issues.md` + `Fixed.md` sibling formats

Run:

```
pandoc docs/Issues.md -o docs/Issues.html --standalone --toc --  
  metadata title="Herald — Issues"  
pandoc docs/Issues.md -o docs/Issues.docx --toc --metadata  
  title="Herald — Issues"  
pandoc docs/Issues.md -o docs/Issues.pdf --pdf-engine=weasyprint  
  --toc --metadata title="Herald — Issues"  
  
pandoc docs/Fixed.md -o docs/Fixed.html --standalone --toc --  
  metadata title="Herald — Fixed"  
pandoc docs/Fixed.md -o docs/Fixed.docx --toc --metadata  
  title="Herald — Fixed"  
pandoc docs/Fixed.md -o docs/Fixed.pdf --pdf-engine=weasyprint --  
  toc --metadata title="Herald — Fixed"  
  
pandoc docs/Status.md -o docs/Status.html --standalone --toc --  
  metadata title="Herald — Status"
```

```
pandoc docs/Status.md -o docs/Status.docx --toc --metadata
  title="Herald - Status"
pandoc docs/Status.md -o docs/Status.pdf --pdf-engine=weasyprint
  --toc --metadata title="Herald - Status"
```



Step 5: Commit

```
git add quickstart/.env.example docs/Issues.md docs/Issues.html
  docs/Issues.docx docs/Issues.pdf docs/Fixed.md docs/
  Fixed.html docs/Fixed.docx docs/Fixed.pdf docs/Status.md
  docs/Status.html docs/Status.docx docs/Status.pdf
git commit -m "HRD-010 step 8: atomic Issues→Fixed migration +
  Status.md r7 + .env.example"
```

HRD-010 row moved Issues.md → Fixed.md atomically per §11.4.19.
Status.md r6→r7: commons_storage now  landed; e2e_bluff_hunt
18 PASS / 0 FAIL. .env.example documents the new HERALD_PG_* and
HERALD_REDIS_* variables.

All siblings (.html/.docx/.pdf) regenerated per §11.4.65.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"

Task 9: Final anti-bluff verification + multi-mirror push

Files: none modified — this task validates and ships.



Step 1: Run the complete anti-bluff battery

Run, in order, and confirm all green:

```
bash tests/test_constitution_inheritance.sh           # expect 15
  PASS / 0 FAIL
bash tests/test_constitution_inheritance_meta.sh      # expect
  META-TEST PASS
bash tests/test_i6_refinement_meta.sh                 # expect 3
  PASS / 0 FAIL
bash tests/test_i8_usability_meta.sh                  # expect 5
  PASS / 0 FAIL
bash scripts/audit_antibluff.sh                       # expect 14
  PASS / 0 FAIL
bash scripts/codegraph_validate.sh                   # expect 7
  PASS / 0 FAIL
bash scripts/e2e_bluff_hunt.sh                       # expect 18
  PASS / 0 FAIL
```

ALL must be green. A FAIL anywhere blocks the push.



Step 2: Re-index CodeGraph for new symbols

Run: `bash scripts/codegraph_setup.sh` (regenerates `.codegraph/codegraph.db` against the new files).

Re-run: `bash scripts/codegraph_validate.sh` Expected: 7+ PASS / 0 FAIL (may grow if `codegraph_validate` probes are updated to include Pool, Queue, Redis symbols — optional improvement).



Step 3: Multi-mirror fan-out push (per Constitution §103)

Run: `git push origin main` Expected: 4 lines showing GitHub + GitLab + GitFlic + GitVerse all received the new commits.



Step 4: Wave 1 partial-close verification

Confirm:

```
git log -10 --oneline
```

Should show 8 new commits (one per Task 1-8) on top of the previous HEAD.

Confirm `docs/Issues.md` no longer contains HRD-010 in its open table. Confirm `docs/Fixed.md` contains HRD-010 in its closed table. Confirm `docs/Status.md` r7 reports `commons_storage`  landed and `e2e_bluff_hunt` 18 PASS.

Self-Review

After completing all tasks, run this checklist:

1. Spec coverage check. Master roadmap declares HRD-010 covers: pgx pool, RLS, migrations, River queue (= `digital.vasic.background`), Redis ACL, pherald migrate. Map:

Roadmap requirement	Task
pgx pool wired through QuickstartBoot	Task 2
RLS context propagation	Task 3 (E14)
Migrations applied at boot	Task 2 (Up() runs RunMigrations)
Background queue extended via <code>digital.vasic.background</code>	Task 5 (E15)
Redis ACL'd via <code>digital.vasic.cache</code>	Task 4 (E16)
<code>pherald migrate</code> subcommand	Task 6
<code>e2e_bluff_hunt.sh</code> +E14/E15/E16	Task 7
HRD row Issues → Fixed	Task 8
Status.md update	Task 8
Multi-mirror push	Task 9

Roadmap requirement	Task
§107 evidence (E14 captured: tenant-A reads only tenant-A's row)	Task 3
§107 evidence (E15 captured: dequeued task ID == enqueued task ID)	Task 5
§107 evidence (E16 captured: key absent after TTL)	Task 4

No coverage gaps.

2. Placeholder scan.

- "HRD-NNN" — appears in honest-stub 501 references; legitimate
- "TBD" — appears only in references to the upstream Catalogue-Check field that must be resolved before the HRD closes; legitimate
- <flavor> — appears in shell command templates; legitimate
- 2026-05-XX — appears in the Fixed.md row template; **engineer MUST replace with actual close date during Task 8**. Not a planning bluff; a deliberate parametric placeholder.

3. Type consistency.

- TaskQueue declared as alias in Task 1; used in Tasks 5; matches upstream `digital.vasic.background.TaskQueue` ✓
- ErrNotBooted declared in Task 1; referenced in Tasks 2/4/5 tests ✓
- Pool() / Queue() / Redis() accessor names consistent across Tasks 1-5 ✓
- Task alias for BackgroundTask declared in Task 1; used in Task 5 ✓
- ParsedDSN + CurrentVersion are added in Task 6's "Engineer note" if they don't exist — flagged for the engineer

4. Anti-bluff trap coverage.

Trap from master roadmap C-tables	Mitigation in this plan
Mock-driven PASS	Every integration test uses live containers; no mocks
Compile-only PASS	Every integration test invokes the resulting API and asserts on its return value
Connection-only PASS	E14/E15/E16 each observe a state mutation (insert row, dequeue exact-ID, TTL-expire)
Empty-field PASS	E14 asserts exact UUID match; E15 asserts exact task-ID match
Catch-all 200 PASS	n/a (no new HTTP routes in this plan)
Gate-bluff PASS	n/a (no new gate invariants — just new e2e probes)
Skipped-silently PASS	E14-E16 SKIP-with-reason if no container runtime; never PASS-by-default
Sandbox-leakage PASS	All tests run inside <code>t.Cleanup(boot.Down)</code> — containers torn down

Execution Handoff

Plan complete and saved to `docs/superpowers/plans/2026-05-20-hrd-010-commons-storage-live-wiring.md`.

Two execution options:

1. **Subagent-Driven (recommended)** — I dispatch a fresh subagent per task, review between tasks, fast iteration. Each subagent gets a clean context window and only the relevant files.
2. **Inline Execution** — Execute tasks in this session using `superpowers:executing-plans`, batch execution with checkpoints for review.

Which approach?